Tasks on K8S-06

1. Create a ConfigMap from a directory containing multiple files and inject the variables into a pod as environment variables.

Objective: Create multiple configuration files inside a directory, create a ConfigMap from that directory, and inject all its values into a Pod as environment variables.

Implementation steps

- **Step 1:** Create a directory
  mkdir config-files
  cd config-files
- **Step 2:** Create multiple files
- Create the first file:
  echo "192.168.0.1" > DB_HOST
- Create the second file:
  echo "admin" > DB_USER
- Create the third file:
  echo "production" > APP_ENV
- Verify the files:
  ls
- **Expected output:**
  APP_ENV DB_HOST DB_USER
- **Step 3:** Create a ConfigMap from the directory
  kubectl create configmap app-config --from-file=.
- **Step 4:** Verify the ConfigMap
  kubectl get configmaps
  kubectl describe configmap app-config
- You should see the three keys: APP_ENV, DB_HOST, and DB_USER.
- **Step 5:** Create a Pod YAML file
  vi configmap-env-pod.yaml
- Add the following configuration:
  apiVersion: v1
  kind: Pod
  metadata:
  name: configmap-env-pod
  spec:
  containers:
    - name: nginx

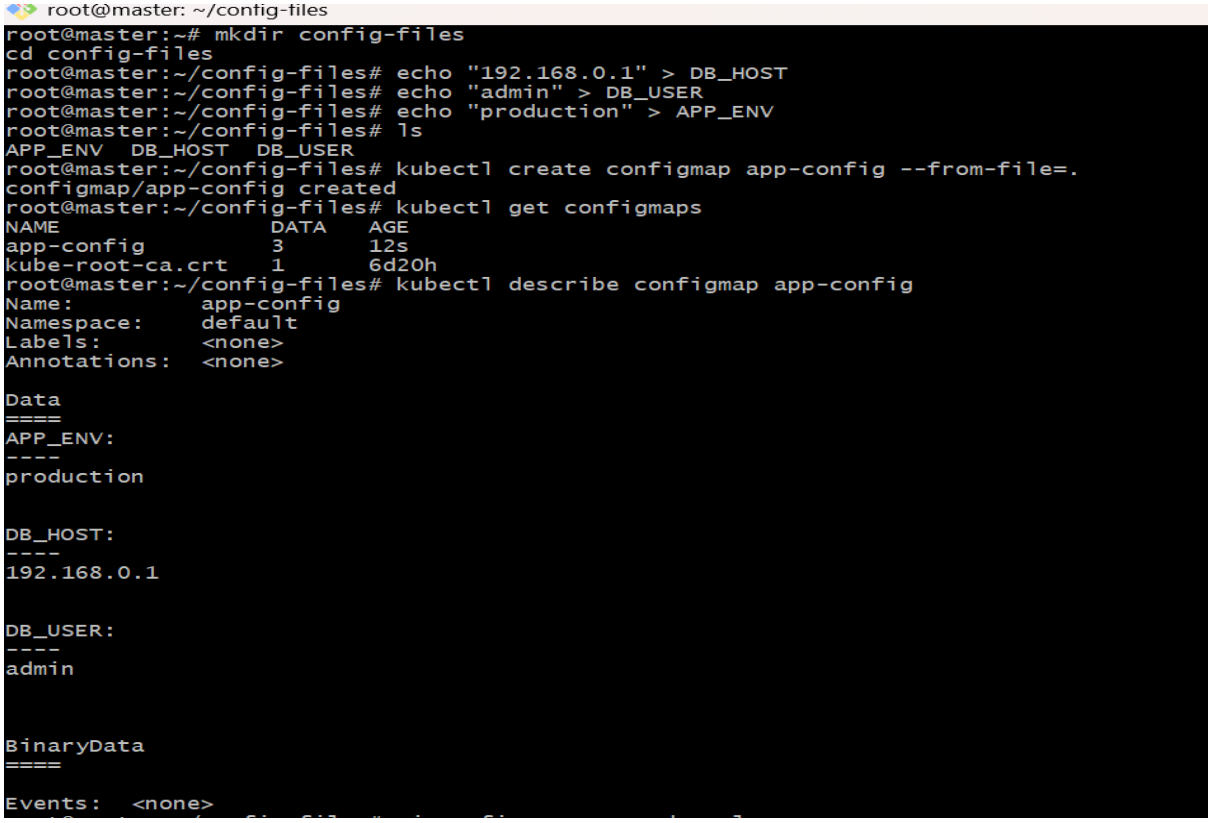
  image: nginx
  envFrom:
    - configMapRef:

  name: app-config
- **Step 6:** Create the Pod
  kubectl apply -f configmap-env-pod.yaml

- **Step 7:** Verify the Pod
  kubectl get pods
- **Step 8:** Verify environment variables inside the Pod
  kubectl exec -it configmap-env-pod -- env
- **Expected output includes:**
  DB_HOST=192.168.0.1
  DB_USER=admin
  APP_ENV=production

Expected result

All three ConfigMap values are available as environment variables inside the Pod.



```
root@master:~/config-files# nano configmap-env-pod.yaml
root@master:~/config-files# cat configmap-env-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-env-pod
spec:
  containers:
    - name: nginx
      image: nginx
      envFrom:
        - configMapRef:
            name: app-config
root@master:~/config-files# kubectl apply -f configmap-env-pod.yaml
pod/configmap-env-pod created
root@master:~/config-files# kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
configmap-env-pod   1/1     Running   0          11s
root@master:~/config-files# kubectl exec -it configmap-env-pod -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=configmap-env-pod
NGINX_VERSION=1.31.8
NJS_VERSION=1.0.2
NJS_RELEASE=1~trixie
ACME_VERSION=0.4.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
DB_USER=admin

APP_ENV=production

DB_HOST=192.168.0.1

KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
TERM=xterm
HOME=/root
root@master:~/config-files#
```

2. Create a ConfigMap from a file and mount it as a volume inside a pod, ensuring the configuration data is available as files.

Objective: Create a ConfigMap from a configuration file and mount it inside a Pod so that the configuration is accessible as a file.

Implementation steps

- **Step 1:** Create a configuration file
  mkdir file-config
  cd file-config
  vi application.properties
- Add the following content:
  username=admin
  password=admin123
  database_ip=192.168.0.1
  database_name=company
- **Step 2:** Create a ConfigMap from the file
  kubectl create configmap application-config --from-file=application.properties
- **Step 3:** Verify the ConfigMap
  kubectl get configmaps
  kubectl describe configmap application-config
- **Step 4:** Create a Pod YAML file
  vi configmap-volume-pod.yaml
- Add the following configuration:

  apiVersion: v1

  kind: Pod

  metadata:

name: configmap-volume-pod

spec:

containers:

name: nginx

image: nginx

volumeMounts:

name: config-volume

mountPath: /etc/app-config

volumes:

name: config-volume

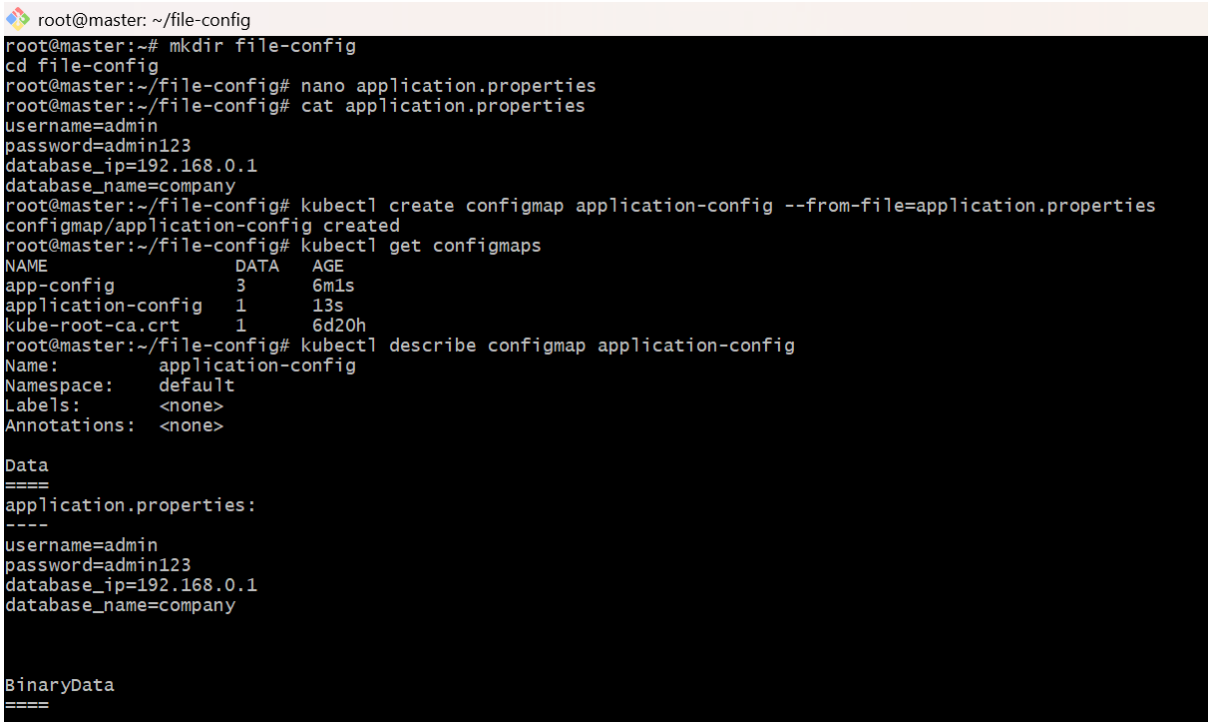
configMap:

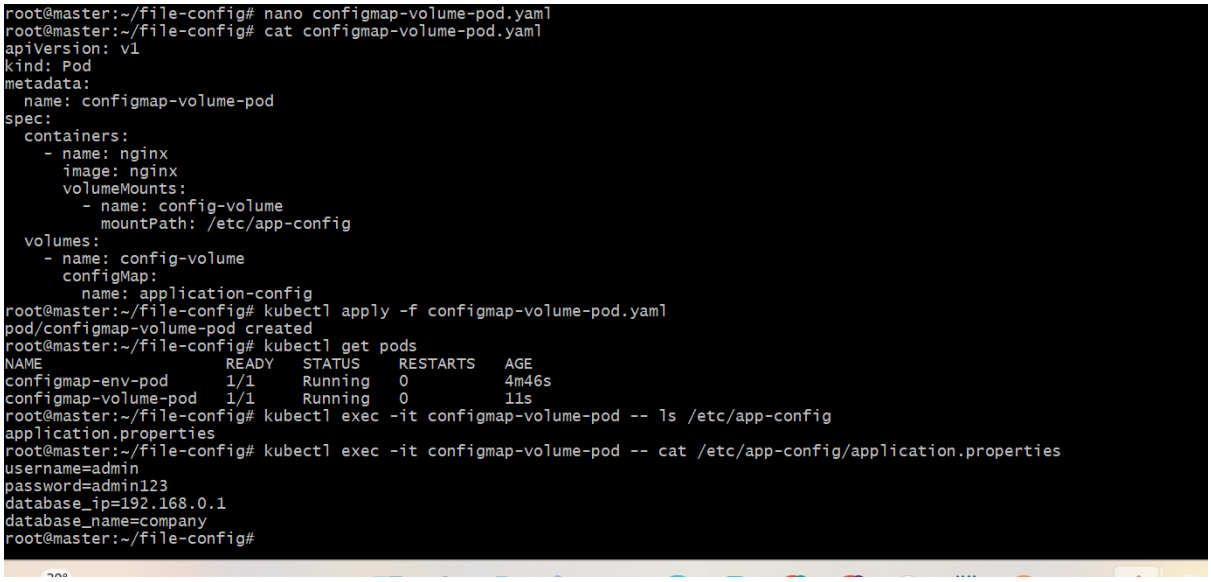
name: application-config

- **Step 5:** Create the Pod
  kubectl apply -f configmap-volume-pod.yaml
- **Step 6:** Verify the Pod status
  kubectl get pods
- **Step 7:** Verify the mounted configuration file
  kubectl exec -it configmap-volume-pod -- ls /etc/app-config
- Expected output:
  application.properties
- **Step 8:** Read the configuration file inside the Pod
  kubectl exec -it configmap-volume-pod -- cat /etc/app-config/application.properties
- **Expected output:**
  username=admin
  password=admin123
  database_ip=192.168.0.1
  database_name=company

Expected result

The ConfigMap is mounted as a volume, and its configuration file is accessible inside the Pod.





3. Create a Secret with sensitive information (username and password) and inject it into a pod as environment variables.

Objective: Create a Kubernetes Secret containing sensitive information and inject the Secret values into a Pod as environment variables.

Implementation steps

- **Step 1:** Create a Secret using literal values
  kubectl create secret generic app-secret \
  --from-literal=username=admin \
  --from-literal=password=admin123
- **Step 2:** Verify the Secret

kubectl get secrets

- **Step 3:** Describe the Secret
  kubectl describe secret app-secret
- You can see the Secret keys and their data sizes, but not their actual values.
- **Step 4:** Create a Pod YAML file
  vi secret-env-pod.yaml
- Add the following configuration:
  apiVersion: v1

  kind: Pod

  metadata:

  name: secret-env-pod

  spec:

  containers:

  name: nginx

  image: nginx

  env:

  name: DB_USERNAME

  valueFrom:

  secretKeyRef:

  name: app-secret

  key: username

  name: DB_PASSWORD

  valueFrom:

  secretKeyRef:

  name: app-secret

  key: password

- **Step 5:** Create the Pod
  kubectl apply -f secret-env-pod.yaml
- **Step 6:** Verify the Pod
  kubectl get pods
- **Step 7:** Verify the environment variables
  kubectl exec -it secret-env-pod -- env
- **Expected output includes:**
  DB_USERNAME=admin
  DB_PASSWORD=admin123

Expected result

The Secret values are injected into the Pod as environment variables.

```
root@master:~# kubectl create secret generic app-secret \
--from-literal=username=admin \
--from-literal=password=admin123
secret/app-secret created
root@master:~# kubectl get secrets
NAME         TYPE     DATA   AGE
app-secret   Opaque   2      12s
root@master:~# kubectl describe secret app-secret
Name:         app-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
password:  8 bytes
username:  5 bytes
root@master:~# nano secret-env-pod.yaml
root@master:~# cat secret-env-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
    - name: nginx
      image: nginx
      env:
        - name: DB_USERNAME
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: username

        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: password
root@master:~# kubectl apply -f secret-env-pod.yaml
pod/secret-env-pod created
```

```
pod/secret-env-pod created
root@master:~# kubectl get pods
NAME                 READY   STATUS    RESTARTS   AGE
configmap-env-pod    1/1     Running   0          8m28s
configmap-volume-pod 1/1     Running   0          3m53s
secret-env-pod       1/1     Running   0          10s
root@master:~# kubectl exec -it secret-env-pod -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=secret-env-pod
NGINX_VERSION=1.31.8
NJS_VERSION=1.0.2
NJS_RELEASE=1~trixie
ACME_VERSION=0.4.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
DB_USERNAME=admin
DB_PASSWORD=admin123
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
TERM=xterm
HOME=/root
root@master:~#
```

4. Create a Secret using a YAML file, mount it as a volume in a pod, and verify the specific Secret values are available as files.

Objective: Create a Secret using a YAML manifest, mount it as a volume, and verify that the Secret values are available as files inside the Pod.

Implementation steps

- **Step 1:** Create a Secret YAML file
  vi app-secret.yaml
- Add the following configuration:

  apiVersion: v1

  kind: Secret

  metadata:

  name: app-file-secret

  type: Opaque

  stringData:

  username: admin

  password: admin123

- Here, stringData allows us to provide plain-text values in the manifest. Kubernetes converts them into the Secret's encoded data representation when storing the object.
- **Step 2:** Create the Secret
  kubectl apply -f app-secret.yaml
- **Step 3:** Verify the Secret
  kubectl get secrets
  kubectl describe secret app-file-secret
- **Step 4:** Create a Pod YAML file
  vi secret-volume-pod.yaml
- Add the following configuration:

  apiVersion: v1

  kind: Pod

  metadata:

  name: secret-volume-pod

  spec:

  containers:

  name: nginx

  image: nginx

  volumeMounts:

  name: secret-volume

  mountPath: /etc/app-secret

  readOnly: true

volumes:

name: secret-volume

secret:

secretName: app-file-secret

- **Step 5:** Create the Pod
  kubectl apply -f secret-volume-pod.yaml
- **Step 6:** Verify the Pod status
  kubectl get pods
- **Step 7:** Verify the Secret files inside the Pod
  kubectl exec -it secret-volume-pod -- ls /etc/app-secret
- **Expected output:**
  password
  username
- **Step 8:** Read the username
  kubectl exec -it secret-volume-pod -- cat /etc/app-secret/username
- **Expected output:**
  admin
- **Step 9:** Read the password
  kubectl exec -it secret-volume-pod -- cat /etc/app-secret/password
- **Expected output:**
  admin123

Expected result

The Secret is mounted as a volume, and both username and password are accessible as separate files inside the Pod.

```
root@master: ~
root@master:~# nano app-secret.yaml
root@master:~# cat app-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-file-secret
type: Opaque
stringData:
  username: admin
  password: admin123
root@master:~# kubectl apply -f app-secret.yaml
secret/app-file-secret created
root@master:~# kubectl get secrets
NAME              TYPE     DATA   AGE
app-file-secret   Opaque   2      11s
app-secret        Opaque   2      3m51s
root@master:~# kubectl describe secret app-file-secret
Name:         app-file-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
password:  8 bytes
username:  5 bytes
root@master:~# nano secret-volume-pod.yaml
root@master:~# cat secret-volume-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-volume-pod
spec:
  containers:
    - name: nginx
      image: nginx
      volumeMounts:
        - name: secret-volume
          mountPath: /etc/app-secret
          readOnly: true
  volumes:
    - name: secret-volume
      secret:
        secretName: app-file-secret
root@master:~# kubectl apply -f secret-volume-pod.yaml
pod/secret-volume-pod created
```

```
root@master:~# kubectl get pods
NAME                  READY   STATUS    RESTARTS   AGE
configmap-env-pod     1/1     Running   0          12m
configmap-volume-pod  1/1     Running   0          7m40s
secret-env-pod        1/1     Running   0          3m57s
secret-volume-pod     1/1     Running   0          12s
root@master:~# kubectl exec -it secret-volume-pod -- ls /etc/app-secret
password  username
root@master:~# kubectl exec -it secret-volume-pod -- cat /etc/app-secret/username
adminroot@mastekubectl exec -it secret-volume-pod -- cat /etc/app-secret/passwordsword
admin123root@master:~#
```

5. Inject a ConfigMap as environment variables and a Secret as files into the same pod, ensuring both are accessible within the pod.

Objective: Create a Pod that uses a ConfigMap for environment variables and a Secret mounted as files. Verify that both configurations are accessible inside the same Pod.

Implementation steps

- **Step 1:** Create a ConfigMap
  kubectl create configmap combined-config \
  --from-literal=APP_ENV=production \
  --from-literal=DB_HOST=192.168.0.1
- **Step 2:** Create a Secret
  kubectl create secret generic combined-secret \
  --from-literal=username=admin \
  --from-literal=password=admin123
- **Step 3:** Verify the ConfigMap and Secret
  kubectl get configmaps
  kubectl get secrets
- **Step 4:** Create a Pod YAML file
  vi combined-pod.yaml
- Add the following configuration:

  apiVersion: v1

  kind: Pod

  metadata:

  name: combined-pod

  spec:

  containers:

  name: nginx

  image: nginx

  envFrom:

  configMapRef:

  name: combined-config

  volumeMounts:

  name: secret-volume

  mountPath: /etc/credentials

  readOnly: true

  volumes:

  name: secret-volume

secret:

secretName: combined-secret

- **Step 5:** Create the Pod
  kubectl apply -f combined-pod.yaml
- **Step 6:** Verify the Pod status
  kubectl get pods
- **Step 7:** Verify ConfigMap environment variables
  kubectl exec -it combined-pod -- env
- **Expected output includes:**
  APP_ENV=production
  DB_HOST=192.168.0.1
- **Step 8:** Verify the Secret files
  kubectl exec -it combined-pod -- ls /etc/credentials
- **Expected output:**
  password
  username
- **Step 9:** Read the Secret username
  kubectl exec -it combined-pod -- cat /etc/credentials/username
- **Expected output:**
  admin
- **Step 10:** Read the Secret password
  kubectl exec -it combined-pod -- cat /etc/credentials/password
- **Expected output:**
  admin123

Expected result

The same Pod successfully accesses ConfigMap values as environment variables and Secret values as mounted files.

```
admin123root@makubectl create configmap combined-config \config \
--from-literal=APP_ENV=production \
--from-literal=DB_HOST=192.168.0.1
configmap/combined-config created
root@master:~# kubectl create secret generic combined-secret \
--from-literal=username=admin \
--from-literal=password=admin123
secret/combined-secret created
root@master:~# kubectl get configmaps
NAME                DATA   AGE
app-config          3      18m
application-config  1      12m
combined-config     2      26s
kube-root-ca.crt    1      6d21h
root@master:~# kubectl get secrets
NAME              TYPE     DATA   AGE
app-file-secret   Opaque   2      4m46s
app-secret        Opaque   2      8m26s
combined-secret   Opaque   2      22s
root@master:~# nano combined-pod.yaml
root@master:~# cat combined-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: combined-pod
spec:
  containers:
    - name: nginx
      image: nginx

      envFrom:
        - configMapRef:
            name: combined-config

      volumeMounts:
        - name: secret-volume
          mountPath: /etc/credentials
          readOnly: true

  volumes:
    - name: secret-volume
      secret:
        secretName: combined-secret
root@master:~#
root@master:~# kubectl apply -f combined-pod.yaml
pod/combined-pod created
```

```
root@master:~# kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
combined-pod        1/1     Running   0          12s
configmap-env-pod   1/1     Running   0          17m
configmap-volume-pod   1/1     Running   0          12m
secret-env-pod      1/1     Running   0          8m43s
secret-volume-pod   1/1     Running   0          4m58s
root@master:~# kubectl exec -it combined-pod -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=combined-pod
NGINX_VERSION=1.31.8
NJS_VERSION=1.0.2
NJS_RELEASE=1~trixie
ACME_VERSION=0.4.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
APP_ENV=production
DB_HOST=192.168.0.1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
TERM=xterm
HOME=/root
root@master:~# kubectl exec -it combined-pod -- ls /etc/credentials
password  username
root@master:~# kubectl exec -it combined-pod -- cat /etc/credentials/username
adminroot@mastekubectl exec -it combined-pod -- cat /etc/credentials/passwordsword
admin123root@master:~#
```